EXPERIMENT1: Vector and Matrix Operations for Deep Learning

Problem Statement: An e-commerce firm represents customers and products as numeric vectors to
power its recommendation engine. Every recommendation is a chain of dot products and matrix multiplications. A small online bookstore has 6 customers and 5 genres. Each customer has a rating
vector, and each book has a genre-weight vector. The store wants to score every customer–book pair
and find similar customers. The objective is to perform vector and matrix operations that underpin a
neural network layer. Use NumPy, Matplotlib.

Tasks:

1. Build the customer–genre (6×5) and genre–book (5×4) matrices in NumPy.
2. Compute dot products, matrix product, transpose, inverse (where valid), and element-wise operations.
3. Compute cosine similarity between customers and identify the two most similar.
4. Show that a dense layer is y = Wx + b by computing it manually for one customer.
5. Visualise the score matrix as a heatmap.

AIM:

To perform vector and matrix operations using NumPy, including dot product, matrix multiplication, transpose and cosine similarity. To understand how these operations are used in neural network computations.

PROGRAM:

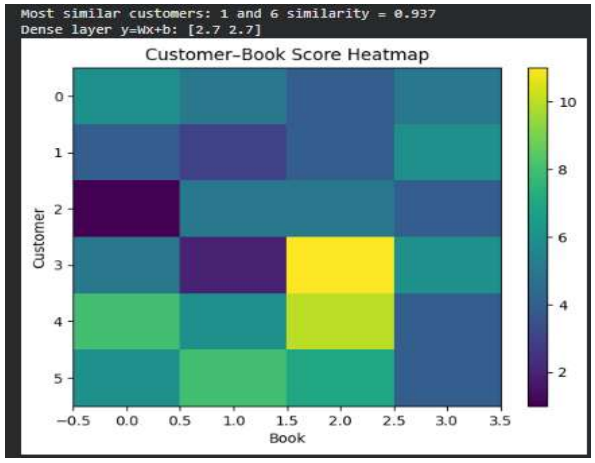
```
customers =
np.array([[5,2,0,1,3],[4,1,2,0,2],[0,5,4,1,0],[1,0,5,4,2],[3,2,1,5,4],[4,3
,0,2,5]], dtype=float)
genre_book = np.array([[1,0,0,1],[0,1,0,0],[0,0,1,1],[1,0,1,0],[0,1,1,0]],
dtype=float)
scores = customers @ genre_book
print("Dot product:", np.dot(customers[0], customers[1]))
print("Matrix product (customer-book scores):\n", scores)
print("Transpose shape:", customers.T.shape)
print("Element-wise square:\n", customers**2)
print("Inverse of valid square matrix:\n",
np.linalg.inv(np.array([[2.,1.],[1.,2.]])))
norm = customers / np.linalg.norm(customers, axis=1, keepdims=True)
sim = norm @ norm.T
print("Cosine similarity:\n", np.round(sim, 2))
np.fill_diagonal(sim, -1)
i,j = np.unravel_index(np.argmax(sim), sim.shape)
```

```
print("Most similar customers:", i+1, "and", j+1, "similarity =",
round(sim[i,j],3))
x = customers[0]; W = np.array([[.2,.1,.3,.4,.2],[.1,.5,.2,.1,.3]])
b = np.array([.5,.2])
print("Dense layer y=Wx+b:", W@x+b)
plt.imshow(scores, aspect='auto'); plt.colorbar(); plt.x label("Book");
plt.ylabel("Customer"); plt.title("Customer-Book Score Heatmap");
plt.show()
```

OUTPUT:

```
Dot product: 28.0
Matrix product (customer-book scores):
 [[ 6.  5.  4.  5.]
 [ 4.  3.  4.  6.]
 [ 1.  5.  5.  4.]
 [ 5.  2. 11.  6.]
 [ 8.  6. 10.  4.]
 [ 6.  8.  7.  4.]]
Transpose shape: (5, 6)
Element-wise square:
 [[25.  4.  0.  1.  9.]
 [16.  1.  4.  0.  4.]
 [ 0. 25. 16.  1.  0.]
 [ 1.  0. 25. 16.  4.]
 [ 9.  4.  1. 25. 16.]
 [16.  9.  0.  4. 25.]]
Inverse of valid square matrix:
 [[ 0.66666667 -0.33333333]
 [-0.33333333  0.66666667]]
Cosine similarity:
 [[1.   0.9  0.27 0.35 0.78 0.94]
 [0.9  1.   0.4  0.53 0.65 0.79]
 [0.27 0.4  1.   0.55 0.4  0.36]
 [0.35 0.53 0.55 1.   0.72 0.44]
 [0.78 0.65 0.4  0.72 1.   0.88]
 [0.94 0.79 0.36 0.44 0.88 1.  ]]
Most similar customers: 1 and 6 similarity = 0.937
Dense layer y=Wx+b: [2.7 2.7]
```

```
Most similar customers: 1 and 6 similarity = 0.937
Dense layer y=Wx+b: [2.7 2.7]
```



RESULT:

Vector and matrix operations were successfully performed. Customer–book scores and cosine similarities were calculated, and the results were visualised using a heatmap.